

Name :Iftikhar Ahmad

RollNo:122652

Topic : Phases of compiler construction

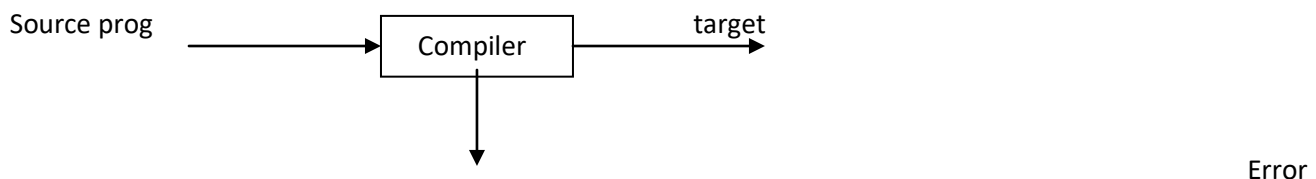
SubmitTo:Muhammed Asgar

SE College Bahawal pur

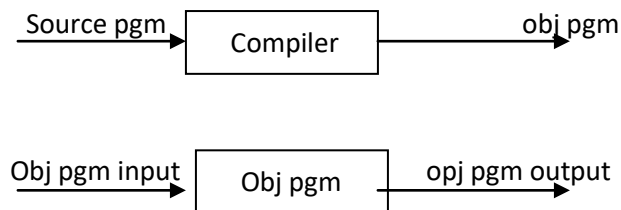
Introduction to compiler

COMPILER

Compiler is a translator program that translates a program written in (HLL) the source program and translate it into an equivalent program in (MLL) the target program. As an important part of a compiler is error showing to the programmer.



Executing a program written n HLL programming language is basically of two parts. the source program must first be compiled translated into a object program. Then the results object program is loaded into a memory executed.

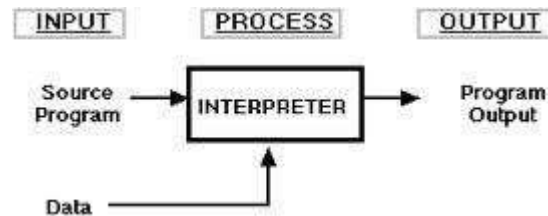


1.4 ASSEMBLER: programmers found it difficult to write or read programs in machine language. They begin to use a mnemonic (symbols) for each machine instruction, which they would subsequently translate into machine language. Such a mnemonic machine language is now called an assembly language. Programs known as assembler were written to automate the

SE college Bahawal Pur Department of CS&IT

translation of assembly language into machine language. The input to an assembler program is called source program, the output is a machine language translation (object program).

1.5 INTERPRETER: An interpreter is a program that appears to execute a source program as if it were machine language.



Languages such as BASIC, SNOBOL, LISP can be translated using interpreters. JAVA also uses interpreter. The process of interpretation can be carried out in following phases.

1. Lexical analysis
2. Syntax analysis
3. Semantic analysis
4. Direct Execution

Advantages:

- Modification of user program can be easily made and implemented as execution proceeds.
- Debugging a program and finding errors is simplified task for a program used for interpretation.
- interpreter for the language makes it machine independent.
-

Disadvantages:

- execution of the program is *slower*.
- Memory consumption is more.

2 Loader and Link-editor:

Once the assembler produces an object program, that program must be placed into memory and executed. The assembler could place the object program directly in memory and transfer control to it, thereby causing the machine language program to be executed. This would waste core by leaving the assembler in memory while the user's program was being executed. Also the programmer would have to retranslate his program with each execution, thus wasting translation time. To overcome these problems of wasted translation time and memory, system programmers developed another component called loader.

"A loader is a program that places programs into memory and prepares them for execution." It would be more efficient if subroutines could be translated into object form by the loader.

could "relocate" directly behind the user's program. The task of adjusting programs so they may be placed in arbitrary core locations is called relocation. Relocation loaders perform four functions.

1.6 TRANSLATOR

A translator is a program that takes as input a program written in one language and produces as output a program in another language. Beside program translation, the translator performs another very important role, the error-detection. Any violation of the HLL specification would be detected and reported to the programmers. Important roles of translator are:

- 1 Translating the hll program input into an equivalent ml program.
- 2 Providing diagnostic messages wherever the programmer violates specification of the hll.

1.7 TYPE OF TRANSLATORS:-

-  INTERPRETOR
-  COMPILER
-  PREPROSESSOR

1.8 LIST OF COMPILERS

1. Ada compilers
- 2 .ALGOL compilers
- 3 .BASIC compilers
- 4 .C# compilers
- 5 .C compilers
- 6 .C++ compilers
- 7 .COBOL compilers
- 8 .D compilers
- 9 .Common Lisp compilers
10. ECMAScript interpreters
11. Eiffel compilers
12. Felix compilers
13. Fortran compilers
14. Haskell compilers
- 15 .Java compilers
16. Pascal compilers
17. PL/I compilers
18. Python compilers

- 19. Scheme compilers
- 20. Smalltalk compilers
- 21. CIL compilers

1.9 STRUCTURE OF THE COMPILER DESIGN

Phases of a compiler: A compiler operates in phases. A phase is a logically interrelated operation that takes source program in one representation and produces output in another representation. The phases of a compiler are shown in below There are two phases of compilation.

- a. Analysis (Machine Independent/Language Dependent)
 - b. Synthesis (Machine Dependent/Language independent)
- Compilation process is partitioned into no-of-sub processes called '**phases**'.

2 Phases of compiler construction

c.

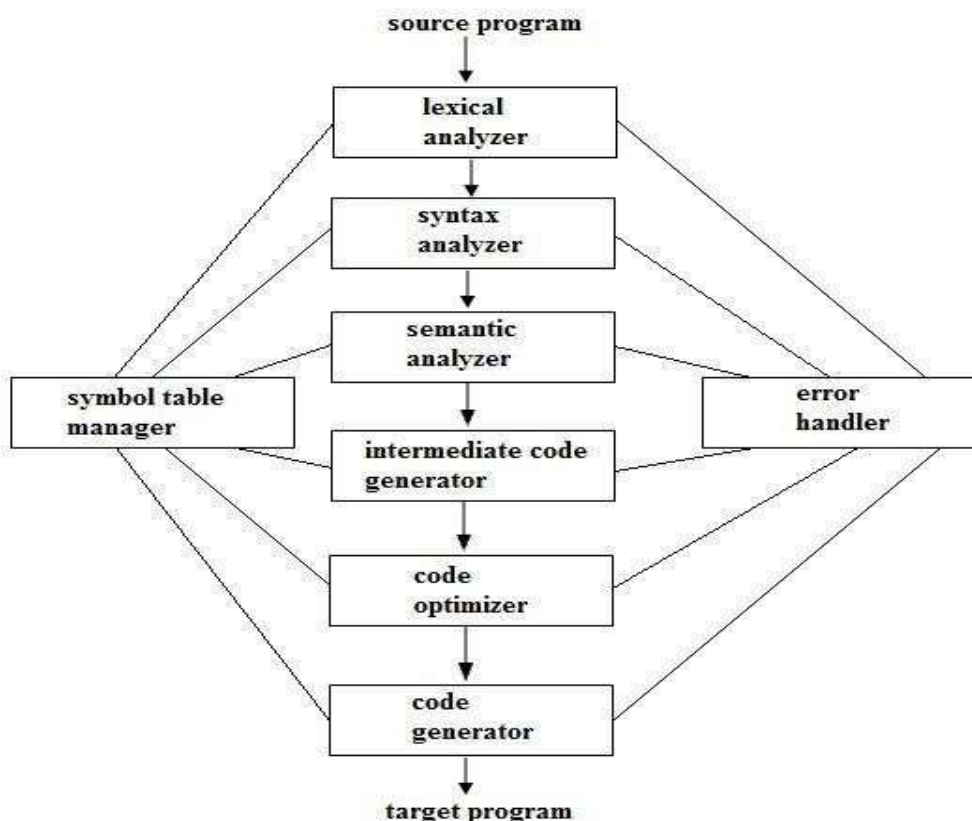


Fig 1.5 Phases of a compiler

Lexical Analysis:-

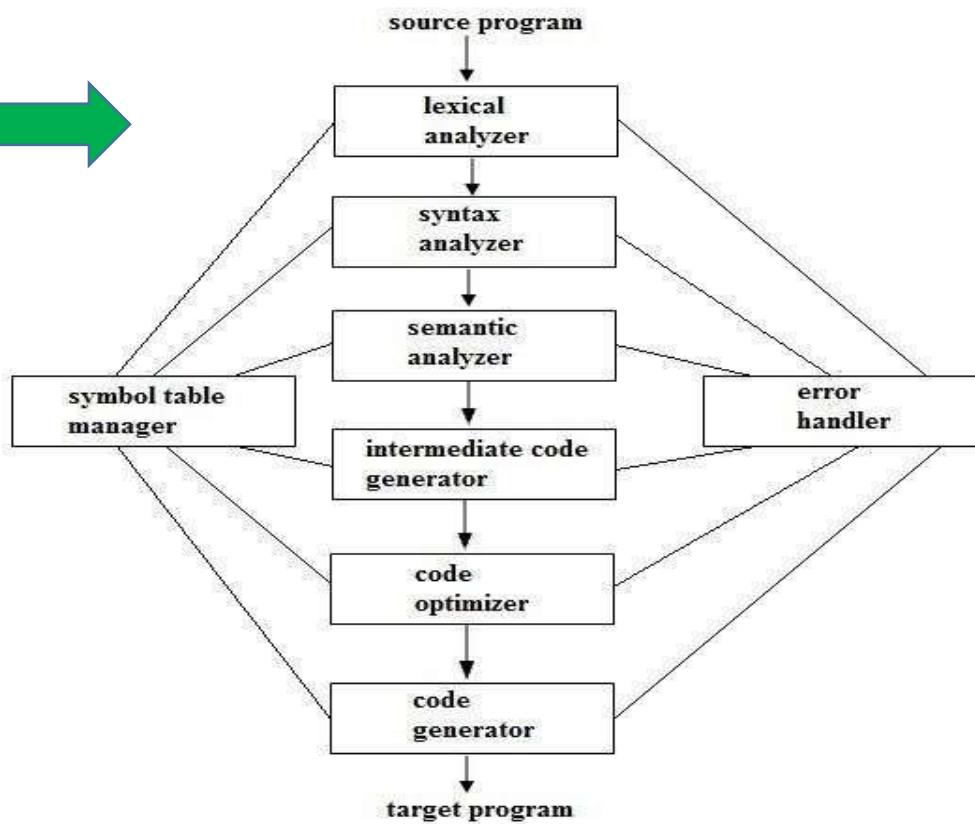


Fig 1.5 Phases of a compiler

Definition

Grouping stream of characters, read from the source program in left-to-right fashion, to form meaningful sequences/components called *lexeme*. This phase is also sometimes called scanning.

Example

For the following assignment statement

*total = tax + quantity * rate*

we get following lexemes

*total, =, tax, +, quantity, * and rate*

Lexical analysis is also responsible to generate output for the scanned lexemes called *tokens* that is passed to the subsequent phase i.e., syntax analysis. These tokens normally takes the following form.

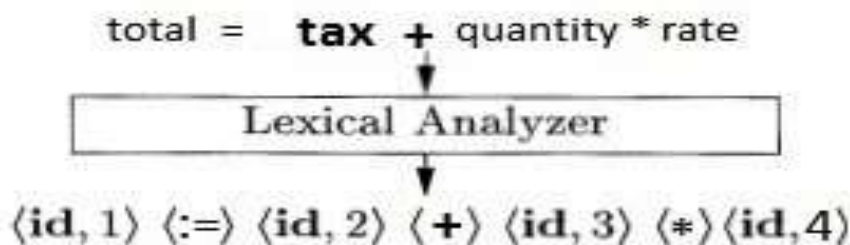
htokenName, attributeValue

where *tokenName* is the abstract symbol to represent the lexeme, and *tokenValue* is token's position in the symbol table.

- Symbol table information is later used by syntax and semantic analysis.

Symbol table

S.N	V.N	V.T
1	total	float
2	tax	float
3	rate	float



Basic action

1. Tokenaziation
2. Remove comment

3. Merge multiline tabs into one
4. Remove blank spaces
5. Longest pattern matching

Syntax analyzer

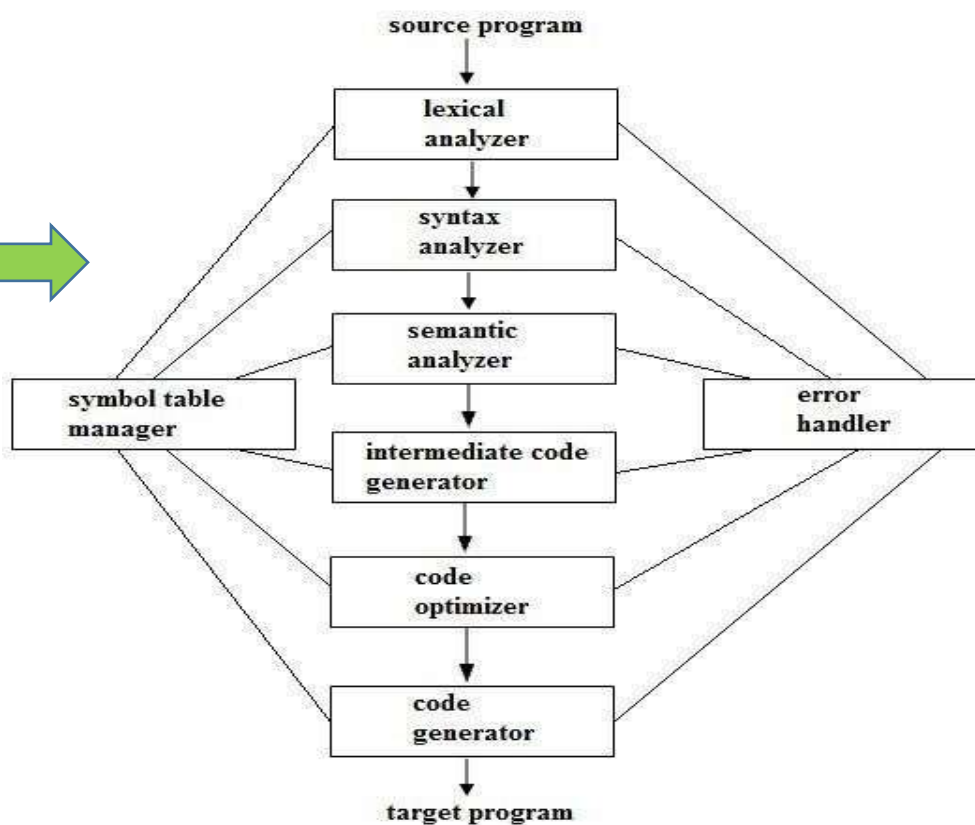


Fig 1.5 Phases of a compiler

The second phase of compiler that is responsible to check the syntax/grammar of the token stream generated by lexical analysis and create a grammatical structure in tree-like representation called *syntax tree*.

In a syntax tree:

Each interior node represents an operation

The children of the node represent arguments/operands.

The syntax tree of our assignment statement example is given bellow that is generated by the token stream of lexical analysis:

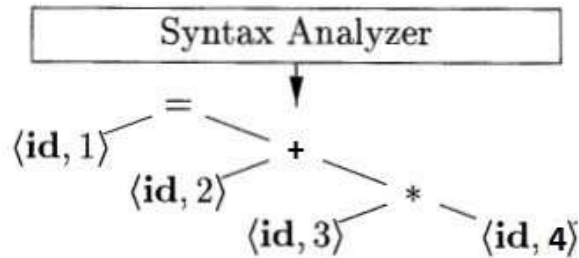


Figure : Syntax Tree for Assignment Statement Example

- The tree above have an interior node labeled '*' with left child as *hid*, 3i and a right child *hid*, 4i.
- *id*, 3i represents the identifier *quantity* and *hid*, 4i represents the identifier *rate*.
- This node also makes it explicit that we must first multiply *quantity* with *rate*.
- Then the node labeled '+' indicates that we must add the result of multiplication to the *tax*.
- Finally, the node labeled '=' indicates that we must store the result of addition into the location for identifier total.
-
-

This ordering is similar to the arithmetic order of operation, where multiplication have higher precedence than addition.

This grammatical structure is then used by subsequent phases to analyze the source program to generate the target program

Semantic analyzer

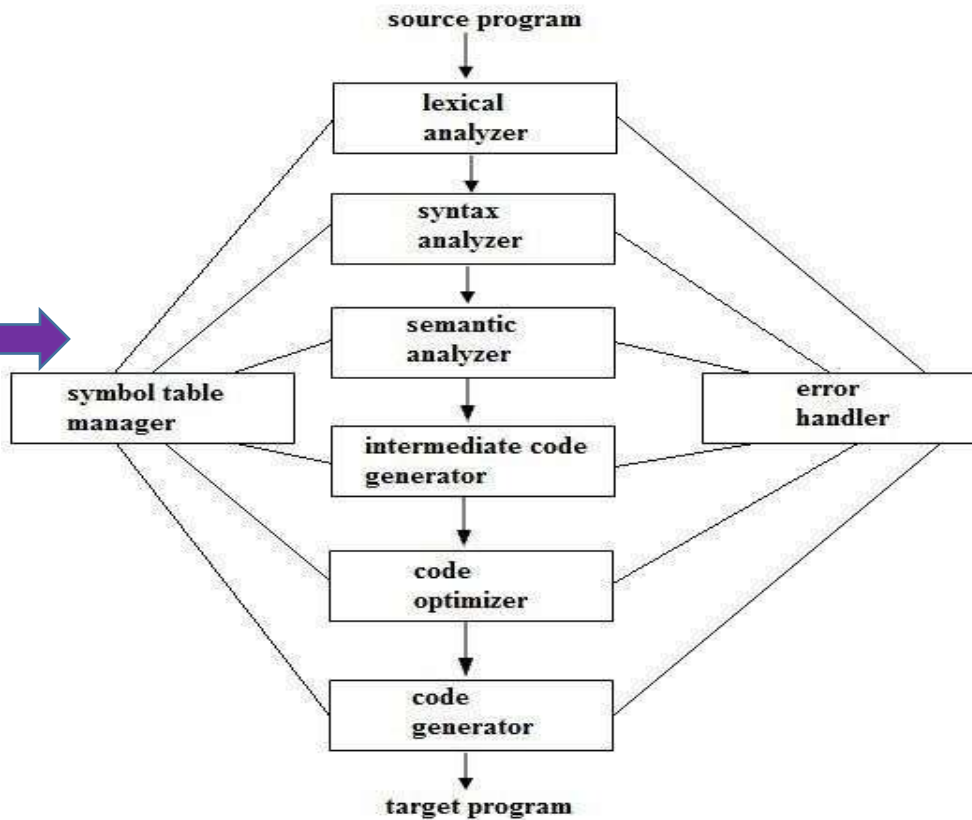


Fig 1.5 Phases of a compiler

The semantic analysis uses syntax tree generated during syntax analysis and the information in the symbol table to check the source program for semantics defined in the language definition. It also maintains type information and store it in either syntax tree or symbol table for further use in the sub-sequent phases.

Type checking is also an important role of this phase where it is checked that each operator have matching operands.

For example, an array index variable need to be integer.

If any error occurs, it must report it that will be called semantic error.

Some language specifications may allow type conversions automatically also called coercions.

Main purpose

1. Type checking

Int +Int

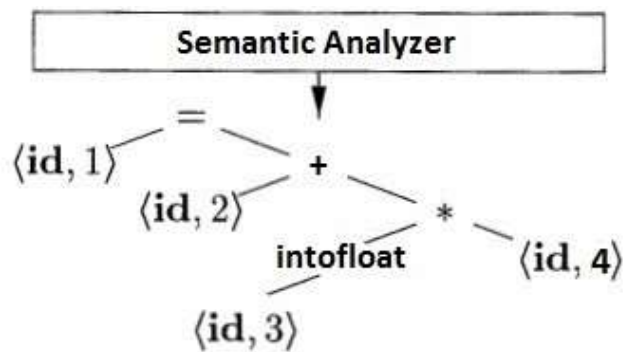
Int +float

Float+ Float

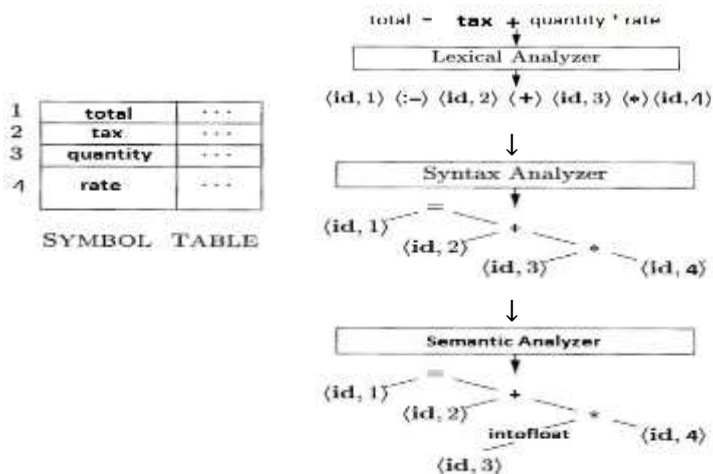
Implicit type casting



Explicit type casting



summary after end of three phase

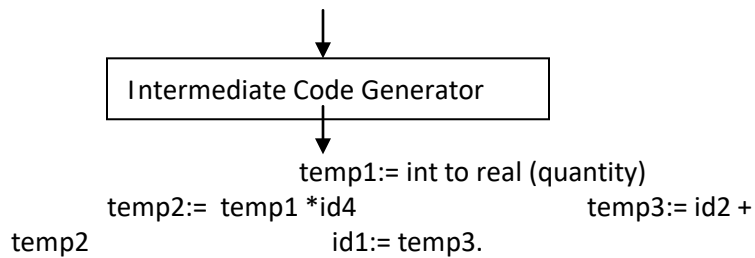


Intermediate Code Generations:-

An intermediate representation of the final machine language code is produced. This phase bridges the analysis and synthesis phases of translation.

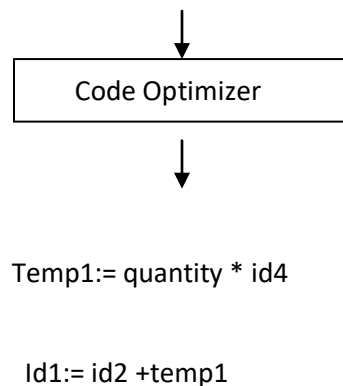
The intermediate code generation uses the structure produced by the syntax analyzer to create a stream of simple instructions. Many styles of intermediate code are possible. One common style uses instruction with one operator and a small number of operands.

The output of the syntax analyzer is some representation of a parse tree. the intermediate code generation phase transforms this parse tree into an intermediate language representation of the source program.



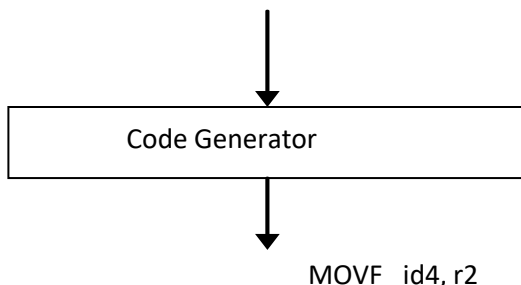
Code optimizer:

This is optional phase described to improve the intermediate code so that the output runs faster and takes less space. Its output is another intermediate code program that does the same job as the original, but in a way that saves time and / or spaces.



Target code generator :

Cg produces the object code by deciding on the memory locations for data, selecting code to access each datum and selecting the registers in which each computation is to be done. Many computers have only a few high speed registers in which computations can be performed quickly. A good code generator would attempt to utilize registers as efficiently as possible.

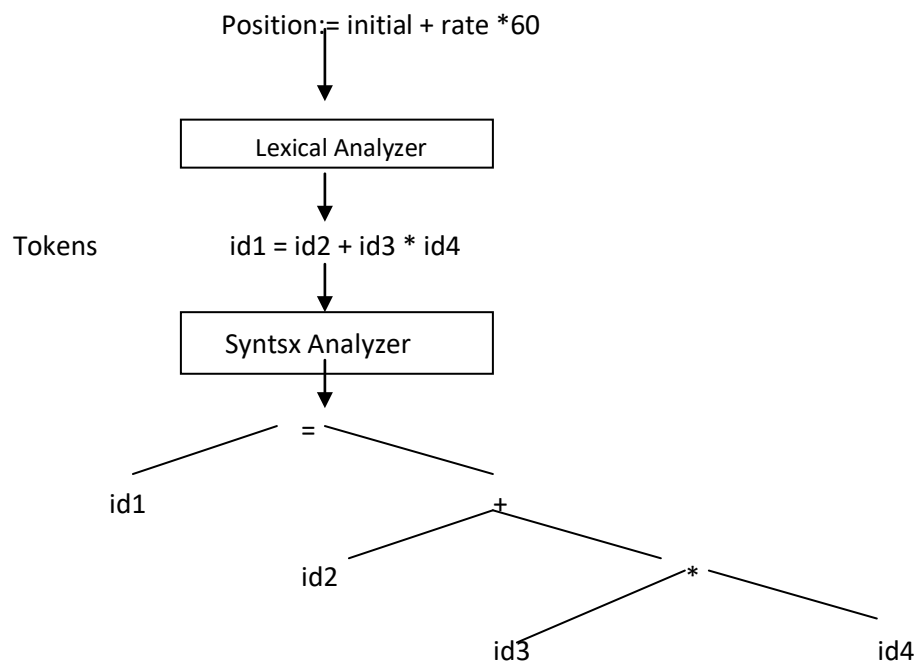


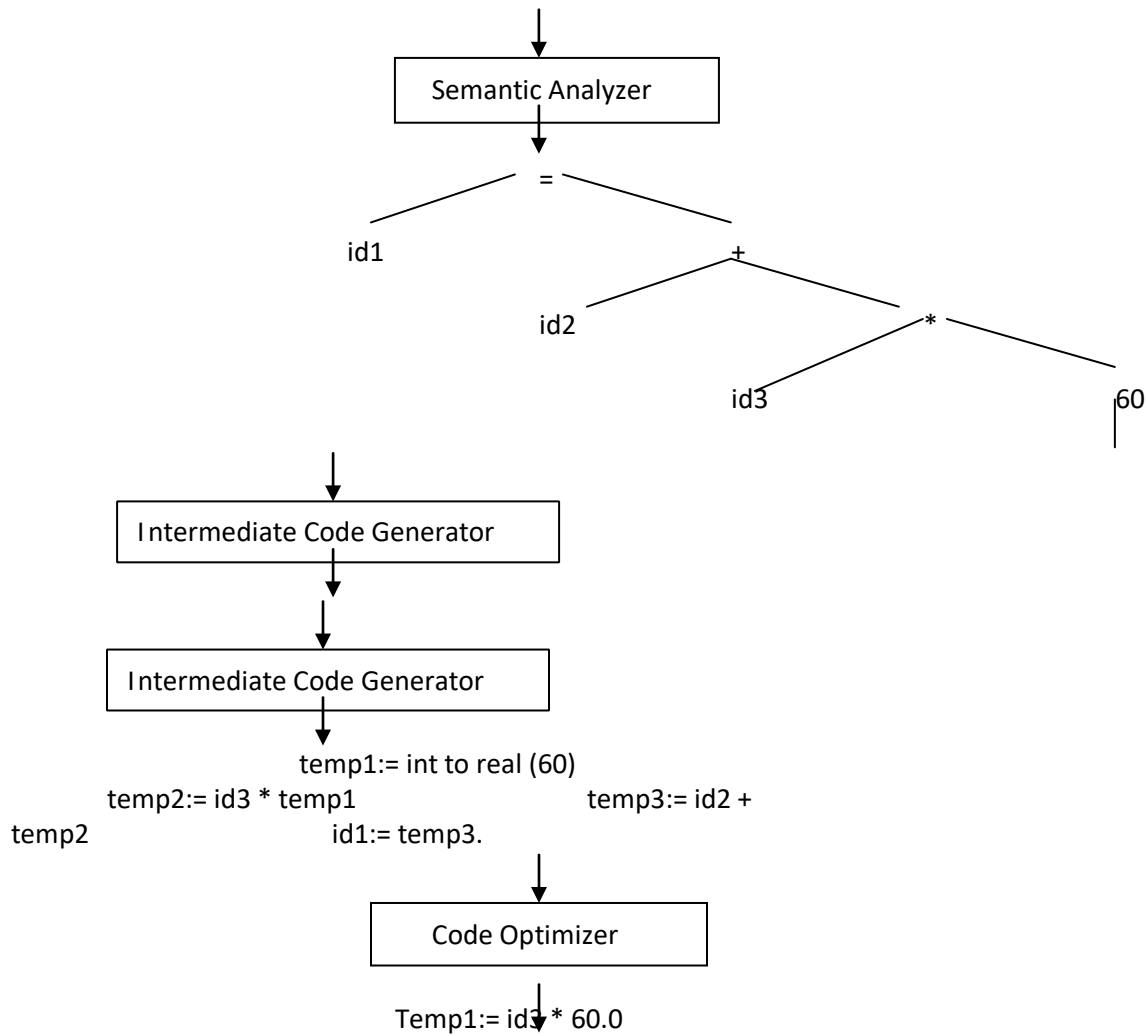
```
MULF  *quantity, r2
MOVF  id2, r1
ADDF  r1, r2, r1
MOVF  r1, id1
```

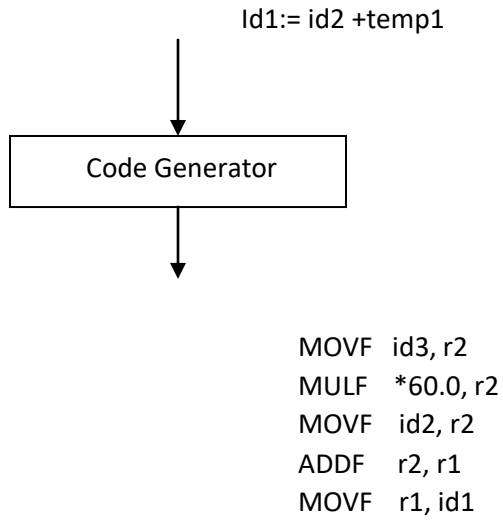
This is complete structure of
compiler construction

There are seven phase of compiler :

Example







Thanks